GNN 기반 가상자산 이상거래 탐지
프로젝트 계획서

XGBoost vs GCN · GraphSAGE · GAT 비교, 그리고 Skip-GCN 하이브리드

HAI 프로젝트 반 · 6인 팀 · 2026.10.28 작성 · 중간점검 10/29 · 최종 발표 11/29

0. 이 문서의 구성

장	내용
1~2장	프로젝트 개요, 데이터
3장 (핵심)	**공통 기반 설계: 데이터 로딩·70/30 분할·전처리·공통 평가 함수·XGBoost 베이스라인 기준** (마린 담당, 11/1 완료)
4~5장	모델별 구현 가이드(GCN·GraphSAGE·GAT), 합치기(하이브리드·SHAP·오류 분석)
6~8장	역할 분담, 타임라인, 협업 규칙
9~10장	리스크와 우선순위, 참고문헌

1. 프로젝트 개요

1.1 문제의식

- 규칙 기반 FDS는 과거 사례로 만든 규칙에 의존하고 거래를 한 건씩 봅니다. 보도에 따르면 은행권 보이스피싱 9,257건 중 FDS 탐지는 1,561건(16.86%)에 그쳤습니다 (금융신문, 2025.10.26).
- 자금세탁처럼 여러 계좌를 거치는 흐름은 거래 사이의 **관계(그래프)**로 봐야 드러납니다.
- 다만 Weber et al.(2019)에서는 Random Forest가 GCN보다 높게 보고되었고, ExSTraQt(ICAIF 2024)는 GNN의 연산 비용·해석 문제를 지적합니다. 따라서 "그래프가 정말 도움이 되는가"를 **공정하게 측정**하는 것 자체가 이 프로젝트의 가치입니다.

1.2 목표와 핵심 질문

> **핵심 질문:** 그래프 구조(GNN)는 탐지 성능을 얼마나, 어떤 비용(속도·해석 가능성)으로 올리는가? GNN과 XGBoost를 합치면 둘의 장점을 얻을 수 있는가?

구분	내용
가설 1	GNN은 XGBoost 단독 대비 PR-AUC·F1에서 [개선 / 비슷 / 열세] 중 하나를 보일 것이다 (결과와 무관하게 이유를 분석).
가설 2	GNN 임베딩을 원본 피처에 결합한 하이브리드(Skip-GCN)는 XGBoost 단독보다 개선될 가능성이 있다.
비교 축	PR-AUC, F1(불법 클래스), 정밀도·재현율, 학습·추론 시간, SHAP 기반 해석

1.3 최종 산출물

- 같은 기준으로 측정한 **4개 모델 비교표** (XGBoost, GCN, GraphSAGE, GAT) + 하이브리드 결과
- SHAP 해석 비교, 시간 단계별·오류 사례 분석
- GitHub 저장소(재현 가능한 코드), 최종 보고서, 발표 자료

2. 데이터: Elliptic Bitcoin Dataset

항목	내용
출처	PyTorch Geometric `EllipticBitcoinDataset` (자동 다운로드) / Kaggle "Elliptic Data Set" / 원 논문 Weber et al. 2019 (arXiv:1908.02591)
규모	노드(거래) 203,769 · 간선(자금 흐름) 234,355 · 시간 단계 49
피처	논문 기준 166개(시간 단계 포함: 로컬 94 + 이웃 집계 72). PyG 버전은 시간 단계를 뺀 165차원으로 알려져 있으므로 **11/1 전에 실제 shape를 확인**합니다.
라벨	불법 4,545 · 정상 42,019 · 미분류 157,205 (불법 약 2%, 정상 약 21%, 미분류 약 77%)
구조 특성	간선은 같은 시간 단계 안의 거래끼리만 연결 (논문 설명 기준) → 시간 단계별로 분리된 그래프 묶음에 가깝습니다.

※ 증권사 계좌 거래 그래프는 비공개이므로, 방법론을 공개 그래프에서 먼저 검증하는 구조입니다. 결과를 "증권사 FDS에 그대로 적용된다"고 주장하지 않고 "방법론 검증"으로 한정합니다.

3. 공통 기반 설계 (마린 담당 · 11/1 완료)

> **왜 중요한가:** 모든 모델이 이 기반 위에서 학습·평가되므로, 분할·지표·측정 방식이 하나라도 다르면 비교가 무의미해집니다. 아래 기준은 **11/1까지 확정해 GitHub에 올리고, 이후에는 바꾸지 않는 것**을 원칙으로 합니다.

3.1 저장소 구조

```
gnn-fds/
├─ README.md              # 실행 방법, 규칙 요약
├─ requirements.txt
├─ data/                  # .gitignore (데이터는 커밋하지 않음)
├─ common/
│  ├─ data.py             # load_data(), get_tabular()
│  ├─ metrics.py          # evaluate(), best_threshold(), evaluate_by_timestep()
│  ├─ utils.py            # set_seed(), Timer, save_row()
│  └─ train_gnn.py        # GNN 공통 학습 루프 (GCN조가 11/5까지 제공)
├─ baselines/xgb_baseline.py
├─ models/gcn.py | sage.py | gat.py
├─ hybrid/skip_gcn.py
├─ analysis/shap_compare.py | error_analysis.py
├─ results/
│  ├─ metrics.csv         # 모든 실험 결과 (한 줄 = 모델·시드·분할)
│  ├─ emb/                # emb_{model}_seed{n}.npy
│  └─ pred/               # pred_{model}_seed{n}.npy
└─ notebooks/ report/
```

3.2 데이터 로딩 기준 (`load_data`)

항목	기준
로딩 방식	`EllipticBitcoinDataset` 사용. 노드 순서는 PyG 기본 순서(txId 오름차순)를 **기준 순서**로 삼고, 임베딩·예측 파일도 이 순서로 저장
라벨	`y` : 0=정상, 1=불법, 2=미분류. **불법(1)을 양성 클래스**로 고정
시간 단계	원본 피처 CSV의 2번째 열에서 읽어 `data.time_step` 으로 추가 (PyG 객체에 없을 경우). 분할에만 사용하고 **모델 입력 피처에서는 제외**
미분류 노드	그래프에는 남겨 두어 GNN 메시지 전달의 이웃으로 사용하되, **손실 계산·평가에서는 제외** (`labeled = y != 2`)
간선	모든 GNN에 `to_undirected` 를 동일하게 적용 (방향 유무는 선택 실험으로만 다룸)
표준화	GNN 입력은 **시간 단계 ≤ 34 노드**의 평균·표준편차로 표준화 (미래 정보 미사용). XGBoost는 원본 값 사용(스케일 불변)
반환값	`data` (x, x_raw, edge_index, y, time_step, train_mask, val_mask, test_mask)

3.3 70/30 분할 기준 (시간 기반)

구간	시간 단계	용도
Train	1 ~ 29	모델 학습
Validation	30 ~ 34	하이퍼파라미터 선택, 조기 종료, 임계값 결정
Test	35 ~ 49	**최종 보고에만 사용** (튜닝 중 확인 금지)

- **시간 순서로 나누는 이유:** 무작위 분할은 미래 정보가 학습에 새어 들어가 성능이 과대평가됩니다. 실제 FDS는 과거로 학습해 미래를 탐지하므로 시간 분할이 현실에 맞습니다.
- **70/30의 의미:** 시간 단계 1~34(34/49 ≈ 69%)를 학습·검증, 35~49를 테스트로 사용합니다. 이는 Weber 논문의 분할(1~34 학습, 35~49 평가)과 같아 **결과를 논문과 직접 비교**할 수 있습니다.
- **최종 학습:** 하이퍼파라미터를 고른 뒤 Train+Validation(1~34)으로 다시 학습하고, 조기 종료로 정해진 반복 횟수를 그대로 사용합니다.
- 분할 후 **구간별 불법·정상 노드 수를 표로 기록**해 README에 남깁니다.

3.4 공통 데이터 함수 스켈레톤

```
# common/data.py  (스켈레톤: 11/1 전에 실제 데이터로 shape·raw_paths 확인 필요)
import numpy as np, pandas as pd, torch
from torch_geometric.datasets import EllipticBitcoinDataset
from torch_geometric.utils import to_undirected

TRAIN_END, VAL_END = 29, 34          # train 1~29, val 30~34, test 35~49

def load_data(root="data/elliptic", standardize=True):
    ds = EllipticBitcoinDataset(root=root)
    data = ds[0]                      # y: 0 정상 / 1 불법 / 2 미분류
    # 시간 단계: 원본 피처 CSV 2번째 열 (PyG 노드 순서 = txId 오름차순)
    raw = pd.read_csv(ds.raw_paths[0], header=None).sort_values(0)
    ts = torch.tensor(raw[1].values, dtype=torch.long)
    assert ts.numel() == data.num_nodes and ts.min() == 1 and ts.max() == 49
    # 정렬 검증: 원본 피처와 PyG x의 앞부분 비교
    assert np.allclose(raw.iloc[:5, 2:].values, data.x[:5].numpy(), atol=1e-4)

    labeled = data.y != 2
    data.time_step = ts
    data.train_mask = labeled & (ts <= TRAIN_END)
    data.val_mask   = labeled & (ts > TRAIN_END) & (ts <= VAL_END)
    data.test_mask  = labeled & (ts > VAL_END)

    data.x_raw = data.x.clone()
    if standardize:                   # 시간 단계 ≤ 34 노드 통계만 사용
        ref = data.x[ts <= VAL_END]
        data.x = (data.x - ref.mean(0)) / (ref.std(0) + 1e-8)
    data.edge_index = to_undirected(data.edge_index)
    return data

def get_tabular(data, mask):          # XGBoost용 (numpy)
    return data.x_raw[mask].numpy(), data.y[mask].numpy()
```

3.5 공통 평가 함수

(1) 지표

지표	정의	쓰는 이유
PR-AUC (주 지표)	불법 클래스의 average precision	불법이 약 2%뿐인 불균형 데이터에서 정확도·ROC-AUC보다 민감. 임계값에 의존하지 않음
F1 (불법) (주 지표)	불법 클래스의 정밀도·재현율 조화평균	Weber 논문과 같은 기준이라 직접 비교 가능
정밀도, 재현율	불법 클래스 기준	오탐(정밀도)과 미탐(재현율)의 균형 확인
ROC-AUC	참고용	일반적 비교용 (불균형에서는 낙관적일 수 있음)
시간 단계별 F1	테스트 시간 단계마다 계산	시간이 지날수록 성능이 어떻게 변하는지(분포 변화) 확인

(2) 임계값 규칙

- **규칙 A:** 임계값 0.5로 계산한 F1·정밀도·재현율을 항상 함께 기록
- **규칙 B:** 검증 구간에서 F1이 최대가 되는 임계값을 찾고(`best_threshold`), 그 값을 테스트에 그대로 적용한 결과도 기록
- 보고서의 대표 수치는 **PR-AUC와 규칙 B의 F1**이며, 테스트 구간에서 임계값을 다시 고르지 않습니다.

(3) 측정 규칙

항목	기준
시드	0, 1, 2, 3, 4 (5회). 결과는 **평균 ± 표준편차**로 보고. GNN이 너무 오래 걸리면 최소 3회
학습 시간	학습 전체 소요 시간(초). GPU 사용 시 `torch.cuda.synchronize()` 후 측정
추론 시간	테스트 노드 전체 예측 시간(초), 5회 반복의 중앙값
환경 기록	장치(CPU/GPU 종류), 파라미터 수. XGBoost는 CPU, GNN은 GPU일 수 있으므로 **장치를 표에 반드시 명시**

```
# common/metrics.py
import numpy as np
from sklearn.metrics import (average_precision_score, roc_auc_score,
                             precision_recall_fscore_support, precision_recall_curve)

def evaluate(y_true, y_prob, threshold=0.5):
    y_pred = (y_prob >= threshold).astype(int)
    p, r, f1, _ = precision_recall_fscore_support(
        y_true, y_pred, average="binary", pos_label=1, zero_division=0)
    return dict(pr_auc=average_precision_score(y_true, y_prob),
                roc_auc=roc_auc_score(y_true, y_prob),
                precision=p, recall=r, f1=f1, threshold=threshold)

def best_threshold(y_true, y_prob):           # 검증 구간에서만 호출
    p, r, t = precision_recall_curve(y_true, y_prob)
    f1 = 2 * p[:-1] * r[:-1] / (p[:-1] + r[:-1] + 1e-12)
    return float(t[np.argmax(f1)])

def evaluate_by_timestep(y_true, y_prob, steps, threshold):
    out = {}
    for s in np.unique(steps):
        m = steps == s
        if y_true[m].sum() > 0:               # 불법이 없는 단계는 건너뜀
            out[int(s)] = evaluate(y_true[m], y_prob[m], threshold)["f1"]
    return out
```

```
# common/utils.py (요약)
import time, random, numpy as np, torch, csv, os

def set_seed(seed):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

class Timer:
    def __enter__(self):
        if torch.cuda.is_available(): torch.cuda.synchronize()
        self.t0 = time.perf_counter(); return self
    def __exit__(self, *a):
        if torch.cuda.is_available(): torch.cuda.synchronize()
        self.sec = time.perf_counter() - self.t0

def save_row(row: dict, path="results/metrics.csv"):   # 한 줄 = 모델·시드·분할
    new = not os.path.exists(path)
    with open(path, "a", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(row));
        if new: w.writeheader()
        w.writerow(row)
```

3.6 XGBoost 베이스라인 기준

항목	기준
입력	원본 피처 전체(시간 단계 제외), 그래프 정보 없음
불균형 처리	`scale_pos_weight = (학습 구간 정상 수) / (학습 구간 불법 수)`
기본 설정	n_estimators 1000(조기 종료 30), learning_rate 0.05, max_depth 6, subsample 0.8, colsample_bytree 0.8, min_child_weight 1, `eval_metric='aucpr'`, `tree_method='hist'`
튜닝	검증 PR-AUC 기준 랜덤 탐색 약 30회: max_depth {4, 6, 8}, learning_rate {0.03, 0.05, 0.1}, min_child_weight {1, 5}, subsample·colsample {0.7, 0.8, 1.0}
최종 학습	선택한 설정으로 Train+Validation(1~34)을 다시 학습 (반복 횟수 = 조기 종료로 찾은 값)
보고 항목	PR-AUC, F1(규칙 A·B), 정밀도·재현율, 학습·추론 시간, 시간 단계별 F1, 피처 중요도
참고 비교	Weber 논문의 Random Forest 보고치(F1 약 0.8 수준, 정확한 값은 논문 표 확인)와 비교해 재현성 점검

```
# baselines/xgb_baseline.py  (스켈레톤)
import xgboost as xgb
from common.data import load_data, get_tabular
from common.metrics import evaluate, best_threshold
from common.utils import set_seed, Timer, save_row

def run_xgb(seed, params):
    set_seed(seed); data = load_data()
    Xtr, ytr = get_tabular(data, data.train_mask)
    Xva, yva = get_tabular(data, data.val_mask)
    spw = (ytr == 0).sum() / (ytr == 1).sum()
    clf = xgb.XGBClassifier(**params, scale_pos_weight=spw, eval_metric="aucpr",
                            early_stopping_rounds=30, tree_method="hist",
                            random_state=seed, n_jobs=-1)
    clf.fit(Xtr, ytr, eval_set=[(Xva, yva)], verbose=False)
    thr = best_threshold(yva, clf.predict_proba(Xva)[:, 1])   # 검증에서 임계값 결정
    # → 이후 Train+Val 재학습(n_estimators = clf.best_iteration + 1), 테스트 평가·저장
```

3.7 결과 저장 형식 (모든 조 공통)

파일	형식
`results/metrics.csv`	컬럼: model, seed, split(val/test), pr_auc, roc_auc, precision, recall, f1, threshold, threshold_type(0.5/val_opt), train_sec, infer_sec, n_params, device
`results/pred/pred_{model}_seed{n}.npy`	(노드 수,) float32, 불법 확률. 노드 순서는 `load_data` 기준 순서
`results/emb/emb_{model}_seed{n}.npy`	(노드 수, 64) float32, 마지막 GNN 층의 노드 임베딩. 노드 순서 동일. model = gcn / sage / gat

3.8 11/1 완료 기준 (Definition of Done)과 코드 리뷰 체크리스트

마린의 11/1 완료 기준

1. `load_data()` 가 오류 없이 동작하고 분할별 노드 수 표를 README에 기록
2. 데이터 검증 통과: 노드 수 203,769 / 시간 단계 1~49 / 라벨 수(불법 4,545·정상 42,019·미분류 157,205) / 피처 차원 확인
3. `evaluate()`, `best_threshold()`, `evaluate_by_timestep()`, `Timer`, `save_row()` 동작 확인
4. XGBoost 베이스라인 5개 시드 결과가 `results/metrics.csv` 에 저장
5. GitHub에 올리고 팀 전체에 사용법 공유 (README 5줄 요약)

#	다른 조 코드를 리뷰할 때 확인할 것
1	데이터·평가는 `common/` 함수만 사용했는가 (직접 구현 금지)
2	테스트 구간을 튜닝·조기 종료·임계값 결정에 쓰지 않았는가
3	시드 고정, 5회(또는 3회) 반복했는가
4	미분류 노드를 손실·평가에서 제외했는가
5	결과·예측·임베딩을 정해진 형식과 노드 순서로 저장했는가
6	시간 단계 피처가 모델 입력에 들어가지 않았는가

4. 모델별 구현 가이드 (GCN조 · SAGE조 · GAT조)

4.1 세 모델의 공통 설정

항목	기준 (세 모델 동일)
구조	GNN 층 2개 + 선형 분류기. **임베딩 = 마지막 GNN 층 출력(64차원)**
손실	가중 교차엔트로피. 클래스 가중치 [1, scale_pos_weight] (XGBoost와 같은 값)
최적화	Adam, lr 0.005, weight_decay 5e-4, 최대 300 에폭
조기 종료	검증 PR-AUC 기준, patience 30
학습 방식	전체 그래프 한 번에 학습(full-batch). 메모리 문제가 있을 때만 `NeighborLoader` 사용
튜닝 범위(조당 8~12회)	lr {0.01, 0.005}, dropout {0.3, 0.5}, weight_decay {5e-4, 1e-3}
최종 학습	Train+Validation(1~34)으로 재학습, 에폭 수는 조기 종료에서 찾은 값
출력	3.7장 형식으로 점수·예측·임베딩 저장

```
# 모델 인터페이스 (모든 조 동일): (로짓, 임베딩)을 반환
class GCN(torch.nn.Module):
    def __init__(self, in_dim, hidden=64, dropout=0.5):
        super().__init__()
        self.c1 = GCNConv(in_dim, hidden)
        self.c2 = GCNConv(hidden, hidden)
        self.head = torch.nn.Linear(hidden, 2)
        self.dp = dropout
    def forward(self, x, edge_index):
        h = F.dropout(F.relu(self.c1(x, edge_index)), self.dp, self.training)
        emb = F.relu(self.c2(h, edge_index))          # 64차원 임베딩
        return self.head(F.dropout(emb, self.dp, self.training)), emb
```

4.2 모델별 특이사항

모델	핵심 아이디어	구현 시 확인할 점
GCN (GCN조)	이웃 노드 특징을 정규화된 평균으로 합침 (Kipf & Welling 2017)	가장 단순하므로 **공통 학습 루프 틀을 먼저 만들어 11/5까지 공유**. `GCNConv`
GraphSAGE (SAGE조)	이웃을 샘플링·집계하고 자기 자신의 표현과 결합 (Hamilton et al. 2017)	`SAGEConv` (aggr=mean 기본). 전체 그래프 학습이 어렵다면 `NeighborLoader`로 미니배치. 추론 시간을 GCN과 비교
GAT (GAT조)	이웃마다 중요도(어텐션)를 학습해 가중 합 (Veličković et al. 2018)	`GATConv`, 헤드 4개×16차원 = 64차원으로 임베딩 크기를 맞춤. 헤드 수 {4, 8} 탐색. 메모리·시간이 가장 클 수 있음

※ 세 모델의 임베딩 차원을 모두 64로 통일해야 하이브리드 비교가 공정합니다.

5. 합치기 (11/12 ~ 11/19)

5.1 Skip-GCN 하이브리드

- 학습된 GNN의 임베딩(64차원)을 원본 피처 옆에 붙여 **XGBoost를 다시 학습**합니다: `X_hybrid = concat([X_raw, emb])`.
- 각 조가 자기 모델의 임베딩으로 만듭니다 (GCN / SAGE / GAT 하이브리드 3종). 비교 공정성을 위해 **XGBoost 설정은 베이스라인과 동일하게 고정**하고, 추가로 재튜닝한 결과는 따로 표시합니다.
- 보조 실험(ablation): ① 임베딩만 입력한 XGBoost ② 원본+임베딩 하이브리드.

> **누수 주의:** GNN은 학습 노드의 라벨로 임베딩을 학습하므로, 같은 학습 노드로 XGBoost를 다시 학습하면 임베딩이 학습 라벨에 과적합된 상태라 하이브리드 성능이 학습 구간에서만 좋아 보일 수 있습니다. 테스트 성능이 기준이며, 학습·검증 성능 차이를 함께 기록합니다. 시간이 남으면 시간 구간을 나눠 임베딩을 만드는 방식(out-of-fold)을 선택 실험으로 시도합니다.

5.2 SHAP 비교 (SAGE조)

- XGBoost 베이스라인과 하이브리드에 `TreeExplainer`를 적용해 상위 20개 피처 기여도를 비교합니다.
- 하이브리드에서 **임베딩 차원 전체가 차지하는 기여도 비율**을 계산합니다. 임베딩 각 차원은 의미가 없으므로 개별 해석 대신 합산해 봅니다.

5.3 오류 분석 (GAT조)

- 시간 단계별 F1 그래프: 특히 **단계 43 전후**의 성능 변화 (Elliptic에서는 특정 시점 이후 새로운 유형의 패턴이 등장해 성능이 크게 떨어진다고 알려져 있어, 직접 확인합니다).
- 오탐·미탐 사례의 특징(연결 수, 이웃의 라벨 분포 등)과 모델별 오류 겹침 비교.

5.4 최종 비교표 (마린·GCN조 취합)

모델	PR-AUC	F1 (규칙 B)	정밀도	재현율	학습 시간	추론 시간	장치
XGBoost	[__]	[__]	[__]	[__]	[__]	[__]	CPU
GCN / SAGE / GAT	[__]	[__]	[__]	[__]	[__]	[__]	GPU
하이브리드 3종	[__]	[__]	[__]	[__]	[__]	[__]	GPU+CPU

모든 값은 5개 시드의 평균 ± 표준편차. 표의 빈칸은 실험 후 채웁니다.

6. 팀 구성과 역할 (6인, 모델별 2인 1조)

조	인원	구현 단계 (~11/12)	합치는 단계 (11/12 ~ 11/19)
GCN조	마린 + 1명	마린: 데이터 로딩·분할·공통 평가 함수·XGBoost 베이스라인 (11/1), 조별 코드 리뷰 파트너: GCN 구현, 공통 학습 루프 틀 제공(11/5)	GCN 하이브리드, **비교표 취합·평가 정리, 보고서 취합**
SAGE조	2명	GraphSAGE 구현·튜닝, 임베딩 저장	SAGE 하이브리드, **SHAP 비교**
GAT조	2명	GAT 구현·튜닝(어텐션 헤드 등), 임베딩 저장	GAT 하이브리드, **오류 분석, 발표 슬라이드**

조 안에서의 역할 나누기

- **구현 담당:** 모델 클래스, 학습 루프 작성
- **실험 담당:** 하이퍼파라미터 탐색, 점수·시간 측정, 임베딩 저장, 학습 곡선 정리 (구현 코드가 돌아가면 바로 시작)
- GCN조는 모델이 가장 쉬우므로 파트너가 구현을 주로 맡고, 마린은 공통 기반·리뷰·XGBoost에 집중

7. 타임라인

기간	단계	마린	각 조
10/29 ~ 11/1	공통 기반 준비	★ **11/1 마감:** 데이터 로딩, 70/30 분할, 공통 평가 함수, XGBoost 베이스라인 → GitHub 공유	논문·PyG 예제 학습, 모델 코드 초안 (PyG 기본 데이터로 시작 가능)
11/2 ~ 11/5	모델 학습 시작	질문 대응, 코드 리뷰, XGBoost 기준점 설명	공통 함수 적용, 첫 학습 (GCN조는 학습 루프 틀 공유)
11/6 ~ 11/12	모델 학습	조별 코드 리뷰, 1차 비교표 틀 준비	튜닝, 점수·시간 측정, 임베딩 저장(emb_*.npy)
11/13 ~ 11/19	합치기	비교표 취합, GCN 하이브리드 지원	조별 하이브리드, SHAP(SAGE조), 오류 분석(GAT조)
11/20 ~ 11/26	마무리	보고서 취합·최종 검수	보고서 파트 작성(GCN조: 비교표·결론, SAGE조: SHAP, GAT조: 오류 분석), 슬라이드 제작
11/27 ~ 11/29	발표	11/27~28 리허설 → 11/29 최종 발표	

목요일 점검 (매주 전원)

날짜	확인할 것
10/29	중간점검 발표, 조 편성·저장소 생성 확인
11/5	공통 기반 사용 확인, 각 조 첫 학습 결과 공유, 막힌 점 해결
11/12	3개 모델 점수·임베딩 완료 여부, 1차 비교표
11/19	하이브리드·SHAP·오류 분석 결과 공유, 부족한 실험 보강 결정
11/26	보고서·발표 자료 확정, 리허설 일정

8. 협업 규칙

- **저장소:** GitHub 하나, `main`은 마린이 관리. 조별 브랜치(`gcn`, `sage`, `gat`)에서 작업 후 Pull Request로 합침. 다른 조 1명이 리뷰(3.8장 체크리스트).
- **공통 기반 변경 금지:** `common/` 수정이 필요하면 마린에게 요청해 전원에게 공지 후 반영 (분할·지표가 중간에 바뀌지 않게).
- **실험 기록:** 모든 결과는 `results/metrics.csv`에 자동 저장. 대시보드식 표는 매주 목요일 전에 갱신.
- **Colab 사용:** 세션이 끊길 수 있으므로 학습 결과는 끝날 때마다 즉시 저장하고, 체크포인트를 남김.
- **의사소통:** 막힌 문제는 30분 이상 혼자 끌지 않고 조원 → 마린 순으로 공유.

9. 리스크와 우선순위

리스크	대응
공통 기반 지연 (11/1)	각 조는 PyG 기본 데이터로 모델부터 작성하고 분할만 나중에 교체. 마린은 GCN조 파트너와 업무 분담
GNN이 XGBoost보다 낮게 나옴	Weber 논문과 같은 방향이므로 정상 시나리오. 원인(시간 분할, 피처 품질, 불균형, 속도)을 분석해 결론으로 사용
하이브리드 누수	5.1장의 주의사항 적용, 학습·테스트 성능 차이 기록, 선택적으로 out-of-fold 실험
시간 단계 43 이후 성능 하락	시간 단계별 F1을 보고하고 오류 분석 항목으로 포함
Colab GPU 제한·세션 끊김	학습 결과 즉시 저장, 체크포인트, 필요 시 `NeighborLoader` 로 메모리 절약
코드·지표 불일치	공통 함수 강제, 코드 리뷰 체크리스트

범위 우선순위

우선순위	내용
필수	XGBoost + GCN·GraphSAGE·GAT 비교 (같은 기준, 3~5개 시드), 최종 비교표, 보고서·발표
권장	하이브리드 3종, SHAP 비교, 시간 단계별 F1
선택	하이브리드 재튜닝, out-of-fold 임베딩, 간선 방향 실험, 오류 사례 심화 분석

10. 참고문헌과 자료

- Weber et al. (2019). *Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics.* arXiv:1908.02591
- Kipf & Welling (2017). *Semi-Supervised Classification with Graph Convolutional Networks.* arXiv:1609.02907
- Hamilton, Ying & Leskovec (2017). *Inductive Representation Learning on Large Graphs.* arXiv:1706.02216
- Veličković et al. (2018). *Graph Attention Networks.* arXiv:1710.10903
- Grinsztajn, Oyallon & Varoquaux (2022). *Why do tree-based models still outperform deep learning on tabular data?* NeurIPS 2022
- ExSTraQt (ICAIF 2024): GNN 기반 거래 이상탐지의 배포 한계와 피처 기반 대안
- 금융신문 (2025.10.26). 은행권 보이스피싱 FDS 탐지율 관련 보도
- PyTorch Geometric 문서: `EllipticBitcoinDataset`, `GCNConv`, `SAGEConv`, `GATConv`, `NeighborLoader`
- 데이터: Kaggle “Elliptic Data Set” (ellipticco/elliptic-data-set)

※ 코드는 스켈레톤이며 PyG 버전에 따라 속성 이름·파일 경로가 다를 수 있습니다. 마린이 11/1 전에 실제 데이터로 검증하고, 달라진 부분은 이 문서의 3장 기준에 맞춰 수정합니다. 수치 표의 [__]는 실험 후 채웁니다.